

شکل ۶-۴ آثار تغییر ترتیب دستورالعمل در راه‌حل پترسن.

همان‌گونه که در بخش‌های بعدی خواهید دید، تنها راه حفظ انحصار متقابل، استفاده از ابزار همگامی مناسب است. بحث ما درباره‌ی این ابزار با پشتیبانی بدوی در سخت‌افزار شروع شده و با APIهای انتزاعی سطح بالای نرم‌افزاری در دسترس برای هر دوی توسعه‌دهندگان هسته و برنامه‌نویسان کاربردی ادامه پیدا می‌کند.

۶-۴ پشتیبانی سخت‌افزاری برای همگامی

ما یک راه‌حل نرم‌افزاری را برای مسئله‌ی بخش بحرانی مطرح کردیم. (ما از آن تحت عنوان راه‌حل ***نرم‌افزاری*** یاد می‌کنیم چون الگوریتم آن هیچ‌گونه پشتیبانی خاصی را از جانب سیستم‌عامل یا دستورالعمل‌های سخت‌افزاری خاصی برای تضمین انحصار متقابل لازم ندارد.) با این وجود، همان‌گونه که بحث شد، راه‌حل‌های نرم‌افزاری تضمینی نیست که در معماری‌های مدرن کامپیوتر کارگر باشند. در این بخش، سه دستورالعمل سخت‌افزاری را ارائه می‌کنیم که پشتیبانی حل مسئله‌ی بخش بحرانی را فراهم می‌کنند. این عملیات بدوی می‌توانند مستقیماً به‌عنوان ابزار همگامی استفاده شوند، یا می‌توانند برای شکل‌دهی بنیان چند راهکار همگامی انتزاعی استفاده شوند.

۶-۴-۱ سدهای حافظه

در بخش ۶-۳ دیدیم که یک سیستم ممکن است ترتیب دستورالعمل‌ها را تغییر دهد، سیاستی که می‌تواند به حالات داده‌ای نامطمئن منجر شود. چگونه یک معماری کامپیوتر تعیین می‌کند چه تضمین‌های حافظه برای یک برنامه‌ی کاربردی فراهم خواهد کرد به **مدل حافظه** معروف است. در حالت کلی، مدل حافظه در یکی از دو دسته‌ی زیر قرار می‌گیرد:

۱- **ترتیب قوی**[1]، که در آن تغییرات حافظه بر روی یک پردازنده بلافاصله برای سایر پردازنده‌ها قابل رؤیت است.

۲- **ترتیب ضعیف**[2]، که در آن، تغییرات حافظه بر روی یک پردازنده ممکن است بلافاصله برای سایر پردازنده‌ها قابل رؤیت نباشد.

مدل‌های حافظه بر اساس نوع پردازنده فرق دارند، لذا توسعه‌دهندگان هسته نمی‌توانند هرگونه فرضی در رابطه با قابلیت دید تغییرات حافظه بر روی یک چندپردازنده‌ی حافظه‌ی

1. Strongly ordered
2. Weakly ordered

مشترک داشته باشند. برای لحاظ این دغدغه، معماری کامپیوترها دستورالعمل‌هایی تدارک می‌بینند که می‌تواند هرگونه تغییرات در حافظه را اجبار کند تا به تمامی پردازنده‌های دیگر منتشر شود، با این که تضمین می‌شود که تغییرات حافظه برای نخ‌های در حال اجرا بر روی سایر پردازنده‌ها قابل رؤیت هستند. این قبیل دستورالعمل‌ها به **سدهای حافظه**[1] یا **حصارهای حافظه**[2] معروف هستند. زمانی که یک دستورالعمل سد حافظه انجام می‌شود، سیستم تضمین می‌نماید که تمامی بارگذاری‌ها و ذخیره‌سازی‌ها قبل از این که عملیات بارگذاری یا ذخیره‌ی بعدی انجام شوند تکمیل می‌شوند. بنابراین، حتی اگر ترتیب دستورالعمل‌ها تغییر کند، سدهای حافظه تضمین می‌دهند که عملیات ذخیره در حافظه تکمیل شده و برای سایر پردازنده‌ها قابل رؤیت هستند قبل از این که عملیات بارگذاری یا ذخیره‌ی آینده انجام شوند.

اجازه دهید به مثال اخیرمان برگردیم که در آن، تغییر ترتیب دستورالعمل‌ها می‌توانست به خروجی اشتباه منجر شود، و از یک سد حافظه استفاده می‌کنیم تا تضمین بدهیم که خروجی مورد انتظار را بدست می‌آوریم.

اگر یک عملیات سد حافظه به نخ ۱ اضافه کنیم

```
while (!flag)
memory_barrier();
print x;
```

ما تضمین می‌کنیم که مقدار flag قبل از مقدار x بارگذاری می‌شود.

با تشابهی، اگر یک سد حافظه میان انتساب‌هایی که توسط نخ ۲ انجام می‌شوند بگذاریم

```
x = 100;
memory_barrier();
flag = true;
```

ما تضمین می‌کنیم که انتساب به x قبل از انتساب به flag اتفاق می‌افتد.

در راه‌حل پترسن می‌توانیم یک سد حافظه میان دو دستورالعمل انتساب اول در بخش ورود بگذاریم تا از تغییر ترتیب عملیات نشان داده شده در شکل ۶-۴ اجتناب شود. توجه کنید که سدهای حافظه عملیات بسیار سطح پایینی در نظر گرفته می‌شوند و نوعاً فقط توسط توسعه‌دهندگان هسته در زمان نوشتن کدهای خاصی که انحصار متقابل تضمینی لازم دارند استفاده می‌شوند.

۶-۴-۲ دستورالعمل‌های سخت‌افزاری

بسیاری از سیستم‌های کامپیوتری مدرن دستورالعمل‌های سخت‌افزاری خاصی فراهم می‌کنند که امکان تست و تغییر محتوای یک کلمه و یا جابه‌جایی محتویات دو کلمه را به صورت **یکپارچه**[3]—یعنی، به صورت یک واحد بی‌وقفه—اجازه می‌دهند. می‌توانیم مسأله‌ی بخش بحرانی را به یک روش نسبتاً ساده با استفاده از این دستورالعمل‌های خاص حل کنیم. به جای

1. Memory barriers
2. Memory fences
3. Atomically

این که راجع به یک دستورالعمل خاص یک ماشین به‌خصوص بحث کنیم، مفاهیم اصلی پشت سر این نوع دستورالعمل‌ها را با بیان دستورالعمل‌های test_and_set() و compare_and_swap() خلاصه می‌کنیم.

دستورالعمل test_and_set() می‌تواند به صورتی که در شکل ۶-۵ نشان داده شده است، تعریف شود. مشخصه‌ی مهم این دستورالعمل، اجرای یکپارچه‌ی آن است. بنابراین، اگر دو دستورالعمل test_and_set() همزمان اجرا شوند (هر کدام در هسته‌ای متفاوت)، به‌ترتیب با ترتیب دلخواهی اجرا خواهند شد. اگر ماشین، دستورالعمل test_and_set() را پشتیبانی کند، در این صورت می‌توانیم انحصار متقابل را با تعریف متغیر منطقی lock، با مقدار اولیه‌ی false پیاده‌سازی کنیم. ساختار پردازه‌ی P_i در شکل ۶-۶ نشان داده می‌شود.

```
boolean test_and_set(boolean *target) {
        boolean rv = *target;
        *target = TRUE;
        return rv;
}
```

شکل ۶-۵ تعریف دستورالعمل test_and_set().

```
do {
        while (test_and_set(&lock)); /* do nothing */

        /* critical section */

        lock = false;

        /* remainder section */
} while (true);
```

شکل ۶-۶ پیاده‌سازی انحصار متقابل با test_and_set().

دستورالعمل compare_and_swap() (CAS)، درست مثل دستورالعمل test_and_set()، به‌صورت یکپارچه بر روی دو کلمه عمل می‌کند، اما از یک راهکار متفاوتی استفاده می‌کند که مبتنی بر مبادله‌ی محتویات دو کلمه است.

دستورالعمل CAS روی سه عملوند کار می‌کند و در شکل ۶-۷ تعریف می‌شود. عملوند value فقط در صورتی که عبارت (*value == expected) به true ارزیابی شود، برابر مقدار new_value انتخاب می‌شود. در هر صورت، compare_and_swap() همواره مقدار اولیه‌ی متغیر value را بر می‌گرداند. مشخصه‌ی مهم این دستورالعمل این است که آن به‌صورت یکپارچه اجرا می‌شود. بنابراین، اگر دو دستورالعمل CAS هم‌زمان اجرا شوند (هر یک بر روی هسته‌ای متفاوت)، آن‌ها به‌ترتیب با یک ترتیب دلخواهی اجرا خواهند شد.

انحصار متقابل با استفاده از CAS می‌تواند به این صورت فراهم شود: متغیر سراسری (lock) اعلان و به ۰ مقداردهی می‌شود. اولین پردازه‌ای که compare_and_swap() را احضار

می‌کند، lock را برابر ۱ انتخاب می‌کند. سپس، وارد بخش بحرانی خود می‌شود، چون مقدار اولیه‌ی lock برابر مقدار مورد انتظار ۰ است. فراخوانی‌های بعدی compare_and_swap() موفق نخواهند بود، چون اکنون lock برابر مقدار مورد انتظار ۰ نیست. موقعی که پردازه از بخش بحرانی خود خارج می‌شود، دوباره lock را ۰ انتخاب می‌کند، که با این کار به پردازه‌ی دیگر اجازه می‌دهد تا وارد بخش بحرانی خود شود. ساختار پردازه‌ی P_i در شکل ۶-۸ نشان داده می‌شود.

```
int compare_and_swap(int *value, int expected, int new_value) {
        int temp = *value;
        if (*value == expected)
        *value = new_value;

        return temp;
}
```

شکل ۶-۷ تعریف دستورالعمل compare_and_swap().

```
do {
        while (compare_and_swap(&lock, 0, 1) != 0); /* do nothing */

        /* critical section */

        lock = 0;

        /* remainder section */

} while (true);
```

شکل ۶-۸ انحصار متقابل با دستورالعمل compare_and_swap().

اگرچه این الگوریتم نیاز انحصار متقابل را ارضا می‌کنند، اما نیاز انتظار محدود را برآورده نمی‌کند. در شکل ۶-۹، الگوریتم دیگری با استفاده از دستورالعمل compare_and_swap() ارائه می‌دهیم که تمامی نیازهای بخش بحرانی را ارضا می‌نماید. ساختمان‌داده‌های مشترک عبارتند از:

```
boolean waiting[n];
boolean lock;
```

عناصر آرایه‌ی waiting به false و lock با 0 مقداردهی می‌شوند. برای اثبات این که نیاز انحصار متقابل برقرار است، توجه داریم که پردازه‌ی P_i فقط در صورتی وارد بخش بحرانی خود می‌شود که waiting[i] == false و یا key == 0 باشد. مقدار key فقط در صورتی می‌تواند 0 شود که compare_and_swap() اجرا شود. اولین پردازه‌ای که compare_and_swap() را اجرا می‌کند key را برابر 0 خواهد یافت؛ سایر پردازه‌ها بایستی منتظر بمانند. تنها در صورتی متغیر waiting[i] می‌تواند false شود که پردازه‌ای دیگر، بخش بحرانی خود را ترک نماید؛ تنها یک

waiting[i]، false انتخاب می‌شود که با این کار، نیاز انحصار متقابل برقرار می‌ماند.

برای اثبات این که، نیاز پیشروی برآورده می‌شود، توجه داریم که استدلال مطرح شده برای انحصار متقابل در اینجا نیز صادق است، چون هر پردازه‌ای که از بخش بحرانی خارج می‌شود، lock را برابر 0 و یا waiting[j] را برابر false انتخاب می‌کند. هر دو به یک پردازه‌ی منتظر امکان می‌دهند تا با پیشروی به بخش بحرانی خود وارد شود.

```
while (true) {
        waiting[i] = true;
        key = 1;
        while (waiting[i] && key == 1)
                key = compare_and_swap(&lock,0,1);
        waiting[i] = false;

                /* critical section */

        j = (i + 1) % n;
        while ((j != i) && !waiting[j])
                j = (j + 1) % n;

        if (j == i)
                lock = 0;
        else
                waiting[j] = false;

                /* remainder section */
}
```

شکل ۶-۹ انحصار متقابل با انتظار محدود با compare_and_swap().

برای اثبات این که نیاز انتظار محدود برآورده می‌شود، توجه داریم که، زمانی‌که پردازه‌ای بخش بحرانی خود را ترک می‌کند، آرایه‌ی waiting را به ترتیب چرخشی (i + ۱، i + ۲، ...، n – ۱، ۰، ...، i – ۱) می‌پیماید. اولین پردازه‌ای را که در این ترتیب، خواهان ورود به بخش بحرانی است (waiting[j] == true)، مشخص کرده و برای نوبت بعدی انتخاب می‌کند. بنابراین، هر پردازه‌ای که بخواهد به بخش بحرانی خود وارد شود حداکثر به تعداد n – ۱ نوبت، منتظر خواهد ماند.

جزئیات کامل مربوط به پیاده‌سازی دستورالعمل‌های test_and_set() و compare_and_swap() یکپارچه در کتاب‌های معماری کامپیوتر مورد بحث قرار می‌گیرند.

۶-۴-۳ متغیرهای یکپارچه

دستورالعمل compare_and_swap() نوعاً مستقیماً برای فراهم‌کردن انحصار متقابل استفاده نمی‌شود. بلکه، به‌عنوان یک بلاک سازنده‌ی اساسی ساخت ابزار دیگر که مسئله‌ی بخش بحرانی

را حل می‌کنند استفاده می‌شود. یک چنین ابزاری، **متغیر یکپارچه**[1] است که عملیات یکپارچه‌ای را بر روی انواع داده‌های مبنایی، همچون مقادیر صحیح و بولی فراهم می‌کند. از بخش ۶-۱ می‌دانیم که افزودن یا کاهش یک‌واحدی یک مقدار صحیح ممکن است یک شرایط مسابقه به‌وجود بیاورد. متغیرهای یکپارچه می‌توانند در تضمین انحصار متقابل در شرایطی استفاده شوند که می‌تواند یک مسابقه‌ی داده بر روی یک متغیر منفرد در حالی که به‌روز می‌شود، به‌وجود بیاید، مثل زمانی که شمارنده‌ای یک واحد افزایش می‌یابد.

اکثر سیستم‌هایی که از متغیرهای یکپارچه پشتیبانی می‌کنند انواع داده‌های یکپارچه‌ی خاصی نیز در کنار توابعی برای دسترسی و دستکاری متغیرهای یکپارچه فراهم می‌کند. این توابع غالباً با استفاده از عملیات compare-_and_swap() پیاده‌سازی می‌شوند. به‌عنوان مثال، دستورالعمل زیر، مقدار صحیح یکپارچه‌ی sequence را یک واحد افزایش می‌دهد:

```
increment(&sequence);
```

که تابع increment() با استفاده از دستورالعمل CAS به‌صورت زیر پیاده‌سازی می‌شود:

```
void increment(atomic_int *v)
{
        int temp;
        do {
                temp = *v;
        }
        while (temp != compare_and_swap(v, temp, temp+1));
}
```

توجه به این نکته مهم است که اگرچه متغیرهای یکپارچه به‌روزرسانی‌های یکپارچه‌ای را فراهم می‌کنند، شرایط مسابقه را به‌صورت کامل در تمامی وضعیت‌ها حل نمی‌کنند. برای مثال، در مسئله‌ی بافر محدود که در بخش ۶-۱ مطرح شد، می‌توانستیم از یک متغیر صحیح یکپارچه برای count استفاده کنیم. این کار تضمین می‌کند که به‌روزرسانی‌های count یکپارچه می‌شوند. با این وجود، پردازه‌های تولیدکننده و مصرف‌کننده نیز حلقه‌های while دارند که شرط آن‌ها به مقدار count وابسته است. شرایطی را در نظر بگیرید که در آن بافر فعلاً خالی است و دو مصرف‌کننده در حلقه هستند در حالی که به‌انتظار count > 0 می‌باشند. اگر تولیدکننده یک آیتم را در بافر وارد کند، هر دو مصرف‌کننده می‌توانند از حلقه‌های while خارج شوند (چون count دیگر برابر ۰ نخواهد بود) و به مصرف روی آورند، حتی اگرچه مقدار count فقط به ۱ مقداردهی شده باشد.

متغیرهای یکپارچه به‌صوررت مرسوم در سیستم‌عامل‌ها در کنار کاربردهای همروند استفاده می‌شوند، هرچند که استفاده‌ی آن‌ها غالباً به به‌روزرسانی‌های تکی داده‌های مشترکی همچون شمارنده‌ها و مولد دنباله‌ها محدود می‌شوند. در بخش بعدی، ابزار قوی‌تری را بررسی می‌کنیم که شرایط مسابقه را در وضعیت‌های کلی‌تری مرتفع می‌سازند.

1. Atomic variable